```
    tracker = APIRequestTracker.new
    tracker.reporter = reporter
    tracker.process(request)
end
```

Every one of these techniques has its uses. Dependency injection is the most common technique we use, and we recommend that you favor it as well. If you'd like to use a library for this task, have a look at the dry-auto_inject project.[6]

Sometimes, dependency injection isn't practical. When you're testing a cluster of objects together, you may not have direct access to the object where you'd like to inject a dependency. Instead, you can stub a factory method or constant.

Watch out, though, for the temptation to treat each difficult situation as a puzzle to solve using more and more powerful RSpec features. When time permits, you'll get better results by refactoring your code to be easier to test with simple techniques.

Of course, refactoring has costs, which may or may not be worth paying for your project. If you do refactor, you'll likely want to get your code under test first. We reach for blunt tools like RSpec's "any instance" feature at times like these, but prefer to use them temporarily. Once we've cleaned up the code, we can drop the crutches and switch to dependency injection.

In these examples, we've been mocking MetricsReporter, a class that belongs to us. In doing so, we've been following the common testing advice, "Only mock types you own." In the next section, we'll see why that admonition is so important.

The Risks of Mocking Third-Party Code

Test doubles are great for providing fake versions of your APIs. Not only do they allow you to test your callers in isolation, they also provide design feedback on your API.

When you try to fake out someone else's API, you miss out on the design benefits of using test doubles. Moreover, you incur extra risks when you mock an interface that you don't own. Specifically, you can end up with tests that fail when they shouldn't, or worse, pass when they shouldn't.

An example will make these risks clearer. The following TwitterUserFormatter class creates a simple string describing a user of the service. Callers will obtain a Twitter::User instance from the Twitter gem (for example, by searching) and pass it to our formatter:[7]

6. http://dry-rb.org/gems/dry-auto_inject/
7. https://github.com/sferik/twitter